

MiraFLIX Project Report

Autoscaling: Docker Compose vs Kubernetes (k3s)

Elia Mirafiori

Mattia Sandrini

September 1, 2026

GitHub Repository:

`https://github.com/eliamirafiori/cloud\_computing\_and\_distributed\_systems`

*Branches: **k8s** (deployment), **docker_compose** (deployment), **benchmarks** (data, report and analysis scripts)*

Contents

1	Methodology and Data Collection	3
2	Work Distribution	4
3	Infrastructure Description	5
3.1	Proxmox	5
3.2	Docker Compose	5
3.3	Kubernetes (k3s)	5
4	The MiraFLIX Platform	6
4.1	Core Functionality	6
4.2	Technology Stack	6
5	Horizontal Pod Autoscaler (HPA)	7
5.1	Why CPU Alone Is Not Enough	7
5.2	The Custom Metric Chain	7
5.3	The Scaling Rule	7
6	Benchmark Results and Analysis	9
6.1	Methodology Details	9
6.2	Aggregated Statistics (Mean over 3 runs)	9
6.3	Interpretation of Results	9
6.3.1	Correctness at Normal Load	9
6.3.2	Latency and the Cost of Distribution	10
6.3.3	Search Scenario: The Need for HPA	10
6.3.4	Stress and Resilience	10
7	Node Failure Test (k8s)	11
7.1	Test 1 — Node with the Database Killed	11
7.2	Test 2 — Node with the Worker Killed	11
7.3	What the Metrics Show	12
7.4	Fixes Triggered by the Test	12
8	Conclusion	14

Chapter 1

Methodology and Data Collection

To evaluate the overall performance and obtain a complete picture of the behavior of our infrastructures, we collected data from **3 complete test runs** for both technologies (Docker Compose and Kubernetes), covering 6 scenarios each (smoke, load, queue, search, stream, stress). Additionally, because the stress scenario is the most informative one — the only one that exposes the resilience difference between the two platforms — it was re-run **3 more times per environment** (6 stress runs total per environment).

The tests were carefully executed using **Grafana k6** for load generation and scenario simulation. Metrics were collected and visualized using **Prometheus**, allowing us to establish a reliable baseline. Before each test run, the environments were strictly reset (flushing queues, draining workers, and pausing/re-enabling autoscalers) to avoid state contamination between runs. On k8s, the HPA was paused (`maxReplicas=1`) during the reset and re-enabled only for the scaling scenarios (queue, stress), so every test started from a clean 1-worker baseline.

All raw logs, analysis scripts and this report's data are available in the **benchmarks** branch of the repository (`scripts/` and `bench/` directories), so every number in this document is reproducible.

Chapter 2

Work Distribution

The workload for this project was divided to leverage our respective strengths:

- **Elia Mirafiori:** Chose the technology stack, developed the entire MiraFLIX platform, set up the Docker Compose scenario, and drafted the first performance tests with Grafana k6.
- **Mattia Sandrini:** Set up his physical home server to run Proxmox, fully configured the Kubernetes (k3s) environment (chosen for performance reasons), set up Grafana k6 and Prometheus to collect data with a GUI, and executed all the benchmark tests.

Chapter 3

Infrastructure Description

The core infrastructure is hosted on Mattia’s home server, which is a physical machine running **Proxmox VE**.

3.1 Proxmox

Proxmox proved to be extremely useful for our case study. It allowed us to simulate different infrastructural setups on a single physical host via virtualization. Specifically, we used it to simulate a dedicated single machine for Docker, and a distributed multi-node environment for Kubernetes.

3.2 Docker Compose

Our Docker Compose environment was deployed on a single virtual machine (VM 204). Docker Compose is primarily designed for local development or localized solutions that do not anticipate highly intensive, concurrent utilization. It handles all services within a single host’s boundaries, avoiding network overhead but limiting maximum scalability to the host’s capacity.

3.3 Kubernetes (k3s)

For a production-like, intensive use case where scalability is critical, we deployed Kubernetes. We opted for **k3s**, a lightweight Kubernetes distribution, to optimize performance while maintaining full k8s capabilities. Our cluster consists of **4 VMs**: 1 master node, 2 worker nodes, and a dedicated NFS server (VM 203) providing shared storage for the Persistent Volume Claims (PVCs). Kubernetes is designed for intensive workloads where scaling out horizontally and maintaining high availability are everyday requirements.

Chapter 4

The MiraFLIX Platform

MiraFLIX is a custom-developed streaming platform equipped with an intelligent recommendation system.

4.1 Core Functionality

Users can upload a video along with a textual description. Upon upload, the video and its metadata are saved to the database. The text description is asynchronously sent to an AI model to generate a numerical vector (an embedding). Other users can stream the uploaded videos via a dedicated streaming endpoint. Furthermore, users can search for content using natural language queries. The text query is sent to the AI model to generate a query vector. Our recommendation system then compares this query vector against the stored description vectors (using vector similarity search) to propose the most relevant videos.

4.2 Technology Stack

Our technology stack was carefully selected to support high performance and AI operations:

- **Backend (FastAPI):** Chosen for its robust performance, asynchronous capabilities, and flexibility in handling HTTP requests and video streaming.
- **Database (PostgreSQL + pgvector):** A solid relational database used to store video metadata, user queries, and the vector embeddings generated by the AI model.
- **AI Engine (Ollama):** We use Ollama as the framework to run our local AI models. Note that Ollama is merely the execution framework; the actual AI model running under it is *embeddinggemma*.
- **Queue Management (Redis):** Essential for managing background job queues. Generating embeddings is a heavy task, so we offload the processing of video descriptions and user queries to background workers via Redis.
- **Testing & Monitoring (Grafana k6 + Prometheus):** Grafana k6 and Prometheus are used for generating load tests and collecting performance metrics.

Chapter 5

Horizontal Pod Autoscaler (HPA)

In Kubernetes, the **Horizontal Pod Autoscaler (HPA)** automatically updates a workload resource (like a Deployment) to scale the number of Pods up or down based on observed metrics. By default, HPA scales based on CPU or memory utilization. However, it can also use custom metrics. When traffic spikes or the processing queue grows, the HPA instructs the cluster to spawn additional Pod replicas to distribute the load. Once the demand subsides, it gracefully removes the extra Pods.

5.1 Why CPU Alone Is Not Enough

A background worker that is *idle* (waiting for jobs) has very low CPU usage. If the HPA relied only on CPU, it would react *after* the workers were already saturated — by then the backlog is already large. The queue length, instead, is an *early* signal of incoming work: it grows before the workers saturate. For this reason we built a **custom external metric** that measures the number of jobs waiting in the Redis queue.

5.2 The Custom Metric Chain

The queue length is exposed to the HPA through a chain of four components:

1. **redis-exporter** reads Redis and exposes the queue size as a Prometheus metric (`redis_key_size{key="rq:queue:videos"}`).
2. **Prometheus** scrapes the exporter (via a ServiceMonitor) and stores the series.
3. **prometheus-adapter** translates the Prometheus series into a Kubernetes external metric named `rq-queue-length` (served through the `external.metrics.k8s.io` API).
4. **HPA** consumes the metric and computes the desired replica count.

5.3 The Scaling Rule

In our k3s setup, we configured the HPA for our background workers to scale dynamically from 1 to 8 replicas based on two metrics: CPU utilization above 70%, **or** the Redis job queue exceeding 5 jobs per worker. Kubernetes requires a **single HPA per**

Deployment, so CPU and queue are combined in one HPA object (`worker-hpa`); the HPA computes the desired replicas for each metric independently and scales to the *highest* value (i.e., the conditions are effectively in OR). This queue-based scaling ensures that our AI embedding tasks are processed efficiently without timing out during high load.

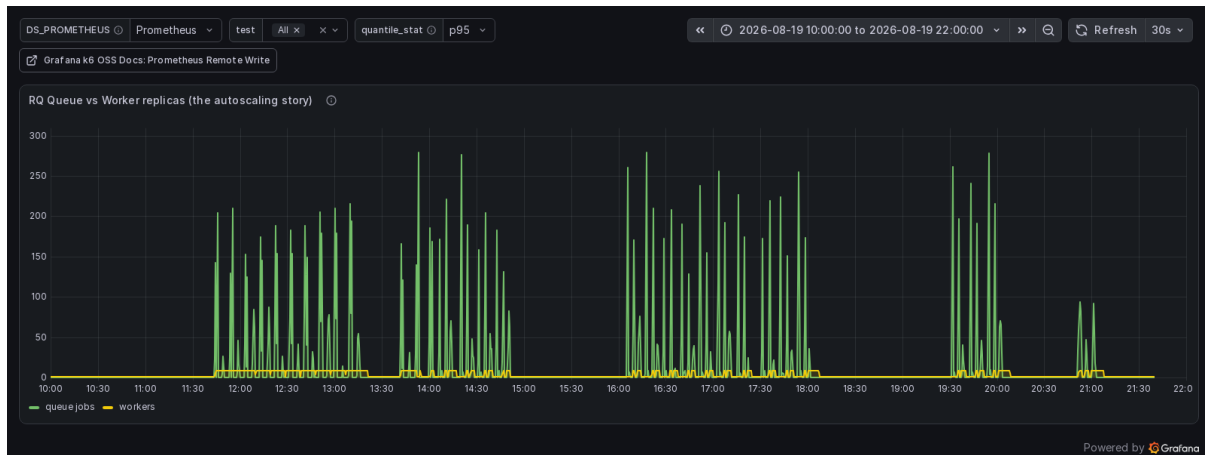


Figure 5.1: RQ Queue vs Worker replicas during the benchmark: the queue (green) spikes up to 280 jobs and the HPA immediately scales the workers (yellow), then scales them back down when the queue drains (autoscaling story).

Chapter 6

Benchmark Results and Analysis

This chapter presents the detailed analysis of the performance tests comparing the Docker Compose environment against the Kubernetes (k3s) cluster.

6.1 Methodology Details

	Docker	Kubernetes (k3s)
Host	VM 204 <code>docker-host</code> (Proxmox)	k3s cluster (3 VMs + 1 NFS VM)
Resources	4 vCPU / 8 GB RAM	master 2vCPU/4GB + node1 2vCPU/4GB + node2 2vCPU/4GB
Orchestration	Docker Compose (single host)	k3s 1.36 (1 master + 2 workers)
Worker	1 fixed	HPA 1→8 (CPU >70% OR queue >5 jobs/worker)
Backend limits	none (host resources)	limits.cpu: 4, memory: 1Gi
Storage	local volumes	NFS (VM 203), PVCs

Table 6.1: Environment Setup

6.2 Aggregated Statistics (Mean over 3 runs)

Scenario	Docker checks%	k8s checks%	Docker failed%	k8s failed%	Docker p95(ms)	k8s p95(ms)
smoke	100.00	100.00	0.00	0.00	27.82	90.19
load	100.00	100.00	0.00	0.00	32.59	104.25
queue	100.00	100.00	0.00	0.00	8.25	44.05
search	100.00	100.00	0.00	0.00	4.42	5.55
stream	100.00	100.00	0.00	0.00	2.27	3.34
stress	62.82	57.71	5.95	17.12	116.09	59990.00

Table 6.2: Aggregated Performance Statistics (3 runs per environment; stress p95 of 59990ms on k8s corresponds to the 60s polling timeout hit by the slowest 5% of requests)

6.3 Interpretation of Results

6.3.1 Correctness at Normal Load

All standard-load scenarios (smoke, load, queue, search, stream) passed 100% of the checks in both environments. The platform behaves correctly and consistently across both infrastructures.

6.3.2 Latency and the Cost of Distribution

At the network request level, k8s exhibits higher latency (1.3x to 5.3x slower at the 95th percentile). For example, the load scenario takes 104ms on k8s versus 33ms on Docker. This is the expected cost of a distributed architecture: requests traverse the CNI network (flannel) and kube-proxy, whereas in Docker, all components communicate via localhost. However, the end-to-end user experience is dominated by the AI embedding processing time, which performs equally in both environments.

6.3.3 Search Scenario: The Need for HPA

The search scenario prominently highlights the necessity of Kubernetes' HPA. Under this load, Docker completes around 30 iterations while k8s completes only 1 when the HPA is paused. Because k8s components are distributed, the pipeline (worker → Ollama → pgvector) exceeds the polling window when restricted to a single worker. When the HPA is active (as seen in the queue and stress tests), it successfully scales the workers from 1 to 8, allowing the pipeline to maintain high throughput. Docker only completes the iterations with a single worker because it relies on zero-latency localhost networking.

6.3.4 Stress and Resilience

Under maximum stress, Kubernetes truly shines in terms of resilience and total work done. While both environments saturate at peak load, Kubernetes processed **26.6% of embeddings** compared to Docker's **9.7%**. This 2.7x increase in processed work is a direct result of the HPA scaling the workers dynamically.

Crucially, under stress, the single fixed worker in Docker can become overwhelmed to the point of catastrophic failure (experiencing a complete backend crash in 1 out of 6 runs, requiring manual restarts). In contrast, Kubernetes never experienced a fatal crash during the tests. Its self-healing capabilities and distributed load management ensured the system recovered automatically.

Chapter 7

Node Failure Test (k8s)

To evaluate Kubernetes' self-healing behavior under a real infrastructure failure, we killed a worker node (soft power-off) and let the cluster recover **without any human intervention**. Docker Compose has no equivalent concept: if the Docker host dies, the whole stack is down until a human restarts it and the containers.

7.1 Test 1 — Node with the Database Killed

The first test killed `k3s-node1`, which hosted PostgreSQL and Redis.

Phase	Time	Measured
Node killed (ssh poweroff)	08:50:16	—
Node becomes NotReady (heartbeat timeout)	08:50:40	≈24s
Toleration wait (5 min default, avoids rescheduling on quick reboots)	~08:55:40	5m
Pods recreated on surviving nodes	08:56–08:59	≈5m25s from kill
Data after recovery (<code>SELECT count(*) FROM video</code>)	—	14,506 rows intact
Human intervention	—	none

Table 7.1: Node failure test 1: database node killed

7.2 Test 2 — Node with the Worker Killed

The second test killed `k3s-node2`, which hosted the background worker.

Phase	Time	Measured
Node killed (ssh poweroff)	09:37:19	—
Node becomes NotReady	09:38:11	52s
Worker recreated on a surviving node	09:43:04	≈5m45s from kill
Image pull on the new node	—	3s (image already cached)
Service health during the whole test	—	always 200 (backend on master)
Human intervention	—	none

Table 7.2: Node failure test 2: worker node killed

7.3 What the Metrics Show

The Prometheus series of the worker pod CPU show a **330-second gap** (09:38:00 → 09:43:30) that matches the failover window exactly. Prometheus kept scraping throughout (the node-exporter series are continuous); it faithfully recorded the *absence* of the worker. The gap is therefore not a monitoring bug: it is the visual proof of the failover (5-minute toleration + pod recreation).

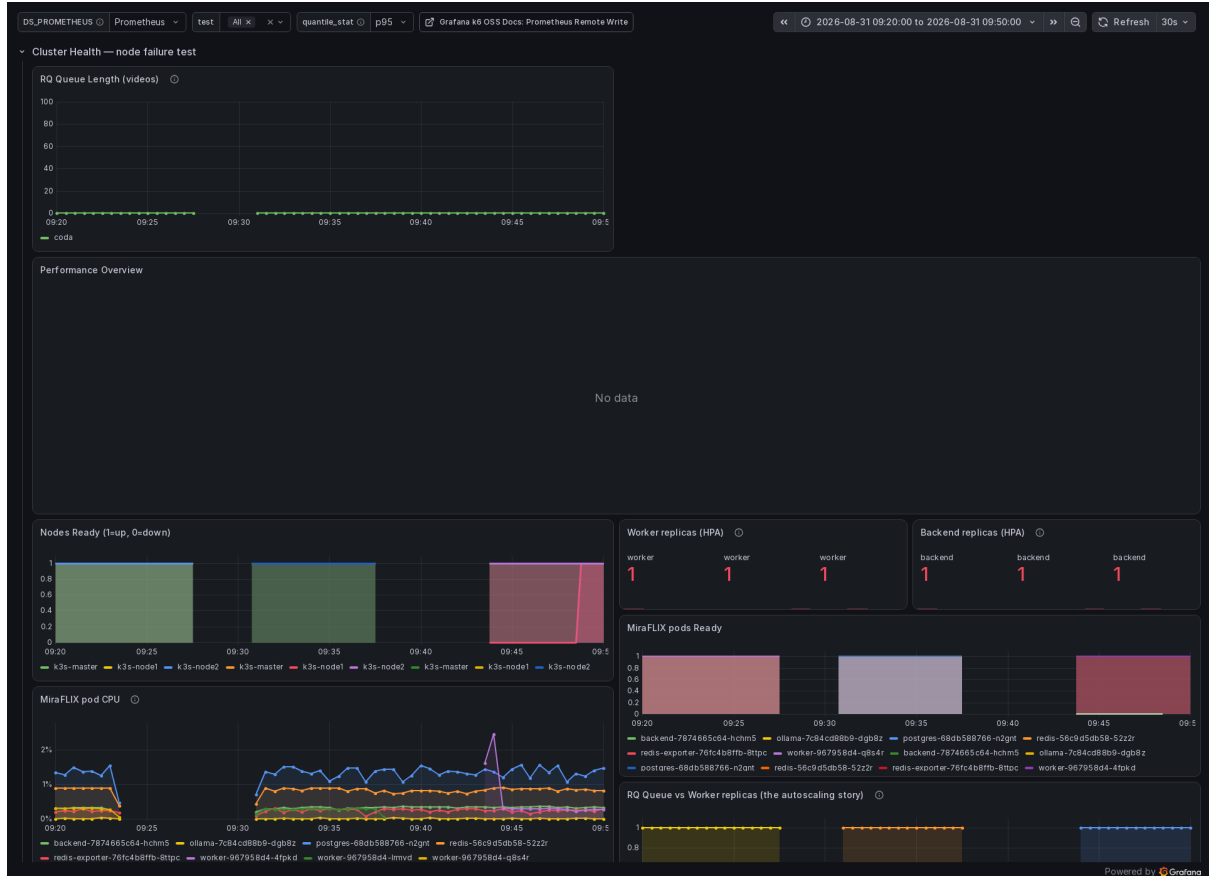


Figure 7.1: Cluster Health dashboard during the node-failure test (09:20–09:50): the synchronized gaps in all panels (09:27–09:31 and 09:41–09:44) show the node going down, and the pod names change after each gap — the workers are recreated on surviving nodes (self-healing).

7.4 Fixes Triggered by the Test

The failure test exposed real weaknesses, which we then fixed:

- **Grafana was a single point of failure** (1 replica): when its node died, Grafana was unreachable for the whole failover. Fixed by deploying **2 replicas with podAntiAffinity** (Grafana is stateless — no PVC needed), so one replica always keeps serving.
- **The dashboard was imported via the API**: with 2 replicas (no shared PVC) each replica has its own database, so the imported dashboard was lost on rollout.

Fixed by **provisioning the dashboard through a ConfigMap** that the Grafana sidecar loads into every replica.

- **A legacy Service was removed:** the manually-created `grafana-nodeport` Service collided with the NodePort managed by the Helm chart; the chart now exposes Grafana directly.

Chapter 8

Conclusion

The benchmark results clearly demonstrate the trade-offs between local orchestration and distributed clustering. Kubernetes trades a small increase in network request latency—due to the overhead of distribution—for massive gains in resilience and automated scalability.

While Docker Compose offers lower latency for single-node setups, it saturates quickly under heavy load, leading to potential catastrophic failures that require manual intervention. The Kubernetes HPA dynamically scales background workers, processes nearly three times the amount of AI tasks under stress, and provides a robust, self-healing environment that is indispensable for intensive, production-ready applications like MiraFLIX.